

Disciplina:

MACHINE LEARNING

Professor: Rafael Barroso



TÓPICO 1

MÉTRICAS DE CLASSIFICAÇÃO: ACURÁCIA, PRECISÃO, RECALL, F1- SCORE E AUC-ROC

MÉTRICAS – CLASSIFICAÇÃO

ACURÁCIA

PRECISÃO

RECALL

F1-SCORE

CURVA ROC

MÉTRICAS – CLASSIFICAÇÃO

MATRIZ DE CONFUSÃO

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

MÉTRICAS – CLASSIFICAÇÃO

ACURÁCIA (ACCURACY)

- A métrica mais intuitiva. Representa a proporção de previsões corretas em relação ao total de previsões.
- Fórmula:

$$Acurácia = \frac{(TP + TN)}{(TP + TN + FP + FN)}$$

- Pontos Positivos:
 - Extremamente fácil de entender e explicar.
- Pontos Negativos:
 - Muito enganosa em datasets desbalanceados.

MÉTRICAS – CLASSIFICAÇÃO

PRECISÃO (PRECISION)

- Responde à pergunta: "De todas as vezes que o modelo previu a classe positiva, quantas ele acertou?" É a métrica do "não quero alarmes falsos".
- Fórmula:

$$Precisão = \frac{TP}{(TP + FP)}$$

- Pontos Positivos:
 - Essencial quando o custo de um **Falso Positivo** é alto.
- Pontos Negativos:
 - Ignora completamente os Falsos Negativos. **Um modelo com alta precisão pode estar deixando de identificar muitos casos positivos.**

MÉTRICAS – CLASSIFICAÇÃO

REVOCAÇÃO/SENSIBILIDADE (RECALL)

- Responde à pergunta: "De todos os casos que eram realmente positivos, quantos o modelo conseguiu identificar?" É a métrica do "não posso deixar passar nada".
- Fórmula:

$$Recall = \frac{TP}{(TP + FN)}$$

- Pontos Positivos:
 - Essencial quando o custo de um **Falso Negativo** é alto.
- Pontos Negativos:
 - Ignora completamente os Falsos Positivos. **Um modelo com recall altíssimo pode estar gerando muitos alarmes falsos.**

MÉTRICAS – CLASSIFICAÇÃO

F1-SCORE

- A média harmônica da Precisão e do Recall. É uma métrica única que busca um equilíbrio entre as duas.
- **Fórmula:**

$$F1 = 2 * \frac{(Precisão * Recall)}{(Precisão + Recall)}$$

- **Pontos Positivos:**
 - Ótimo para comparar modelos quando você precisa de um balanço entre Precisão e Recall.
- **Pontos Negativos:**
 - Menos intuitivo de explicar. A perda da interpretabilidade individual de precisão e recall pode ser uma desvantagem.

MÉTRICAS – CLASSIFICAÇÃO

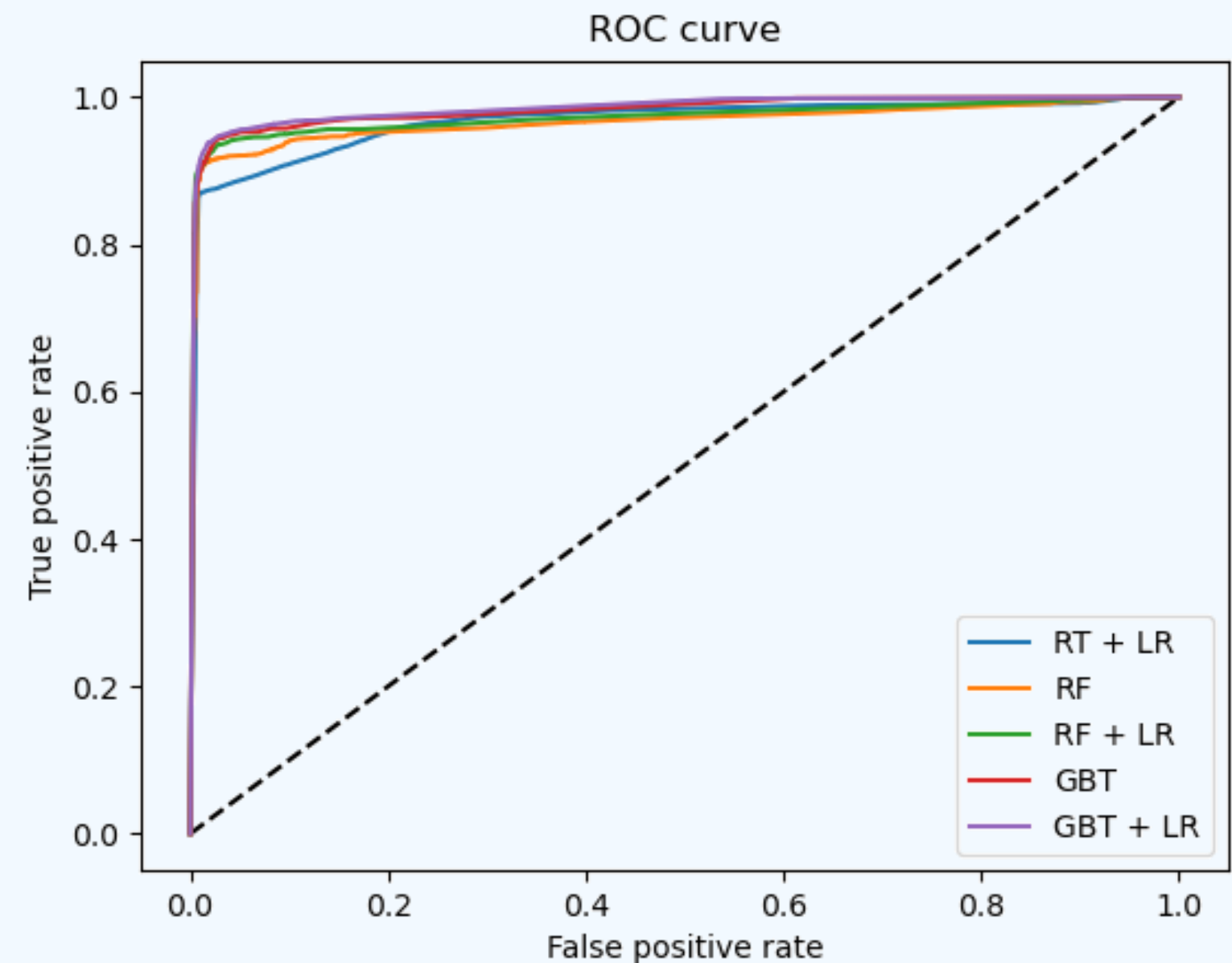
CURVA ROC/AUC

- Uma visualização gráfica que mostra a performance de um modelo de classificação em todos os limiares de classificação. Ela plota a **Taxa de Verdadeiros Positivos** contra a **Taxa de Falsos Positivos**.
- **Intepretação:**
 - A maioria dos classificadores fornece uma probabilidade (ex: 0.85 de chance de ser "doença"). A curva ROC é criada ao variar o limiar de decisão (ex: considerar >0.5 como positivo, depois >0.6 , >0.7 , etc.) e plotar os resultados.
 - Um modelo excelente tem uma curva que "sobe" rapidamente e fica no canto superior esquerdo.
 - A linha diagonal representa um modelo aleatório ("chute").
 - **Interpretação do AUC:** A probabilidade de que o modelo classifique uma amostra positiva aleatória com uma pontuação maior do que uma amostra negativa aleatória.

MÉTRICAS – CLASSIFICAÇÃO

CURVA ROC/AUC

- Pontos Positivos:
 - Excelente para visualizar o *trade-off* do modelo.
 - Não é afetada pelo desbalanceamento das classes.



MÉTRICAS – CLASSIFICAÇÃO

Trabalho

sklearn.metrics.classification_rep

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

scikit-learn 1.3.1

Please cite us if you use the software.

sklearn.metrics.classification_report

classification_report

Examples using

sklearn.metrics.classification_report

sklearn.metrics.classification_report

sklearn.metrics.classification_report(y_true, y_pred, *, labels=None, target_names=None, sample_weight=None, digits=2, output_dict=False, zero_division='warn')
[source]

Build a text report showing the main classification metrics.

Read more in the User Guide.

Parameters:

y_true : 1d array-like, or label indicator array / sparse matrix

Ground truth (correct) target values.

y_pred : 1d array-like, or label indicator array / sparse matrix

Estimated targets as returned by a classifier.

labels : array-like of shape (n_labels,), default=None

Optional list of label indices to include in the report.

target_names : array-like of shape (n_labels,), default=None

Optional display names matching the labels (same order).

sample_weight : array-like of shape (n_samples,), default=None

Sample weights.

digits : int, default=2

Number of digits for formatting output floating point values. When output_dict is True, this will be ignored and the returned values will not be rounded.

output_dict : bool, default=False

If True, return output as dict.

New in version 0.20.

zero_division : {'warn', 0.0, 1.0, np.nan}, default='warn'

Sets the value to return when there is a zero division. If set to 'warn', this acts as 0, but warnings are also raised.

New in version 1.3: np.nan option was added.

Returns:

report : str or dict

Text summary of the precision, recall, F1 score for each class. Dictionary returned if output_dict is True. Dictionary has the following structure:

```
{ 'label 1': { 'precision':0.5,
              'recall':1.0,
              'f1-score':0.67,
              'support':1},
  'label 2': { ... },
  ...
}
```

The reported averages include macro average (averaging the unweighted mean per label), weighted average (averaging the support-weighted mean per label), and sample average (only for multilabel classification). Micro average (averaging the total true positives, false negatives and false positives) is only shown for multi-label or multi-class with a subset of classes, because it corresponds to accuracy otherwise and would be the same for all metrics. See also precision_recall_fscore_support for more details on averages.

Trabalho

sklearn.metrics.roc_auc_score

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html

scikit-learn 1.3.1

Please cite us if you use the software.

sklearn.metrics.roc_auc_score

roc_auc_score

Examples using

sklearn.metrics.roc_auc_score

sklearn.metrics.roc_auc_score

sklearn.metrics.roc_auc_score(y_true, y_score, *, average='macro', sample_weight=None, max_fpr=None, multi_class='raise', labels=None)
[source]

Compute Area Under the Receiver Operating Characteristic Curve (ROC AUC) from prediction scores.

Note: this implementation can be used with binary, multiclass and multilabel classification, but some restrictions apply (see Parameters).

Read more in the User Guide.

Parameters:

y_true : array-like of shape (n_samples,) or (n_samples, n_classes)

True labels or binary label indicators. The binary and multiclass cases expect labels with shape (n_samples,) while the multilabel case expects binary label indicators with shape (n_samples, n_classes).

y_score : array-like of shape (n_samples,) or (n_samples, n_classes)

Target scores.

- In the binary case, it corresponds to an array of shape (n_samples,). Both probability estimates and non-thresholded decision values can be provided. The probability estimates correspond to the **probability of the class with the greater label**, i.e. estimator.classes_[1] and thus estimator.predict_proba(X, y)[:, 1]. The decision values corresponds to the output of estimator.decision_function(X, y). See more information in the User guide;
- In the multiclass case, it corresponds to an array of shape (n_samples, n_classes) of probability estimates provided by the predict_proba method. The probability estimates **must** sum to 1 across the possible classes. In addition, the order of the class scores must correspond to the order of labels, if provided, or else to the numerical or lexicographical order of the labels in y_true. See more information in the User guide;
- In the multilabel case, it corresponds to an array of shape (n_samples, n_classes). Probability estimates are provided by the predict_proba method and the non-thresholded decision values by the decision_function method. The probability estimates correspond to the **probability of the class with the greater label for each output** of the classifier. See more information in the User guide.

average : {'micro', 'macro', 'samples', 'weighted'} or None, default='macro'

If None, the scores for each class are returned. Otherwise, this determines the type of averaging performed on the data. Note: multiclass ROC AUC currently only handles the 'macro' and 'weighted' averages. For multiclass targets, average=None is only implemented for multi_class='ovr' and average='micro' is only implemented for multi_class='ovr'.

'micro':

Calculate metrics globally by considering each element of the label indicator matrix as a label.

'macro':

Calculate metrics for each label, and find their unweighted mean. This does not take label imbalance into account.

'weighted':

Calculate metrics for each label, and find their average, weighted by support (the number of true instances for each label).

'samples':

Calculate metrics for each instance, and find their average.

TÓPICO 2

MÉTRICAS DE REGRESSÃO: MAE, MSE, RMSE E R^2

MÉTRICAS – REGRESSÃO

MSE

RMSE

MAE

MAPE

R^2

MÉTRICAS – REGRESSÃO

MSE – ERRO QUADRÁTICO MÉDIO

- Calcula a média dos erros elevados ao quadrado.
- **Fórmula:**

$$MSE = \frac{\sum (y_i - \hat{y}_i)^2}{n}$$

- **Pontos Positivos:**
 - Útil matematicamente para otimização de modelos. **Penaliza fortemente erros grandes.**
- **Pontos Negativos:**
 - Sensível a outliers (pontos com erro muito grande).
 - Difícil de interpretar devido à unidade ao quadrado.

MÉTRICAS – REGRESSÃO

RMSE – RAIZ DO ERRO QUADRÁTICO MÉDIO

- É simplesmente a raiz quadrada do MSE.
- **Fórmula:**

$$RMSE = \sqrt{\frac{\sum (y_i - \hat{y}_i)^2}{n}}$$

- **Pontos Positivos:**
 - Uma das métricas mais comuns e úteis para regressão.
 - Interpretação direta na unidade original.
- **Pontos Negativos:**
 - Continua sendo mais sensível a outliers do que o MAE.

MÉTRICAS – REGRESSÃO

MAE – ERRO ABSOLUTO MÉDIO

- Calcula a média dos erros absolutos (sem sinal).
- Fórmula:

$$MAE = \frac{\sum |y_i - \hat{y}_i|}{n}$$

- Pontos Positivos:
 - Fácil de interpretar e robusta a outliers (um erro grande não é "explodido" como no MSE).
- Pontos Negativos:
 - Não diferencia bem entre muitos erros pequenos e poucos erros grandes.

MÉTRICAS – REGRESSÃO

MAE – ERRO ABSOLUTO MÉDIO

- Calcula a média dos erros absolutos (sem sinal).
- Fórmula:

$$MAE = \frac{\sum |y_i - \hat{y}_i|}{n}$$

- Pontos Positivos:
 - Fácil de interpretar e robusta a outliers (um erro grande não é "explodido" como no MSE).
- Pontos Negativos:
 - Não diferencia bem entre muitos erros pequenos e poucos erros grandes.

MÉTRICAS – REGRESSÃO

MAPE – ERRO PERCENTUAL ABSOLUTO MÉDIO

- Similar ao MAE, mas calcula o erro em termos percentuais.
- Fórmula:

$$MAPE = \frac{100}{n} * \sum \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

- Pontos Positivos:
 - Excelente para comunicar o resultado, pois é expresso em porcentagem.
- Pontos Negativos:
 - Não pode ser usada se o valor real (y_i) puder ser zero (divisão por zero).

MÉTRICAS – REGRESSÃO

R^2 – COEFICIENTE DE DETERMINAÇÃO

- Diferente das outras, o R^2 não mede o erro. Ele mede a **proporção da variância na variável alvo que é explicada pelo modelo**.

Compara com o erro caso se use a média.

- **Fórmula:**

$$R^2 = 1 - \left(\frac{\text{Erro do Modelo}}{\text{Erro do Modelo Base}} \right) \text{ ou } 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - y_{bar})^2}$$

- **Pontos Positivos:**

- Fornece uma medida relativa e padronizada do quão bem o modelo se ajusta aos dados.

- **Pontos Negativos:**

- Um valor alto de R^2 não garante que o modelo é bom (*overfit*).

- Seu valor sempre aumenta ao adicionar mais variáveis ao modelo, mesmo que sejam inúteis (use o **R^2 Ajustado** para corrigir isso).

MÉTRICAS – REGRESSÃO

sklearn.dummy

sklearn.ensemble

sklearn.exceptions

sklearn.experimental

sklearn.feature_extraction

sklearn.feature_selection

sklearn.frozen

sklearn.gaussian_process

sklearn.impute

sklearn.inspection

sklearn.isotonic

sklearn.kernel_approximation

sklearn.kernel_ridge

sklearn.linear_model

sklearn.manifold

sklearn.metrics

check_scoring

get_scorer

get_scorer_names

make_scorer

accuracy_score

auc

average_precision_score

balanced_accuracy_score

brier_score_loss

class_likelihood_ratios

classification_report

cohen_kappa_score

confusion_matrix

d2_log_loss_score

API Reference

sklearn.metrics

sklearn.metrics

Score functions, performance metrics, pairwise metrics and distance computations.

User guide. See the [Metrics and scoring: quantifying the quality of predictions](#) and [Pairwise metrics, Affinities and Kernels](#) sections for further details.

Model selection interface

User guide. See the [The scoring parameter: defining model evaluation rules](#) section for further details.

check_scoring	Determine scorer from user options.
get_scorer	Get a scorer from string.
get_scorer_names	Get the names of all available scorers.
make_scorer	Make a scorer from a performance metric or loss function.

Classification metrics

User guide. See the [Classification metrics](#) section for further details.

accuracy_score	Accuracy classification score.
auc	Compute Area Under the Curve (AUC) using the trapezoidal rule.
average_precision_score	Compute average precision (AP) from prediction scores.
balanced_accuracy_score	Compute the balanced accuracy.

TÓPICO 3

VALIDAÇÃO CRUZADA: K-FOLD E STRATIFIED K-FOLD

VALIDAÇÃO CRUZADA (CV)

- Quando treinamos e testamos um modelo usando uma única divisão de dados, nosso resultado de performance pode ser muito dependente da "sorte".
- CV é uma técnica de reamostragem usada para avaliar modelos de em um conjunto de dados limitado. A ideia é treinar e testar o modelo múltiplas vezes em diferentes subconjuntos dos seus dados.
- O objetivo é uma estimativa mais estável e confiável da performance do modelo em dados que ele nunca viu, reduzindo o impacto da aleatoriedade na divisão dos dados.

VALIDAÇÃO CRUZADA (CV)

K-FOLD

K-FOLD
ESTRATIFICADO

LEAVE-ONE-
OUT

VALIDAÇÃO CRUZADA (CV)

K-FOLD

▪ Funcionamento:

- Divide-se o conjunto de dados em K subconjuntos. O modelo é treinado em K-1 subconjuntos e avaliado no subconjunto restante. Isso é repetido K vezes, com cada subconjunto sendo usado como conjunto de teste uma vez.

▪ Pontos Positivos:

- Fornece uma estimativa de performance muito mais robusta do que uma única divisão treino-teste.
- Utiliza todos os dados tanto para treino quanto para teste ao longo das iterações.
- É menos suscetível a erros de *overfitting* do que a avaliação em um único conjunto de dados.

▪ Pontos Negativos:

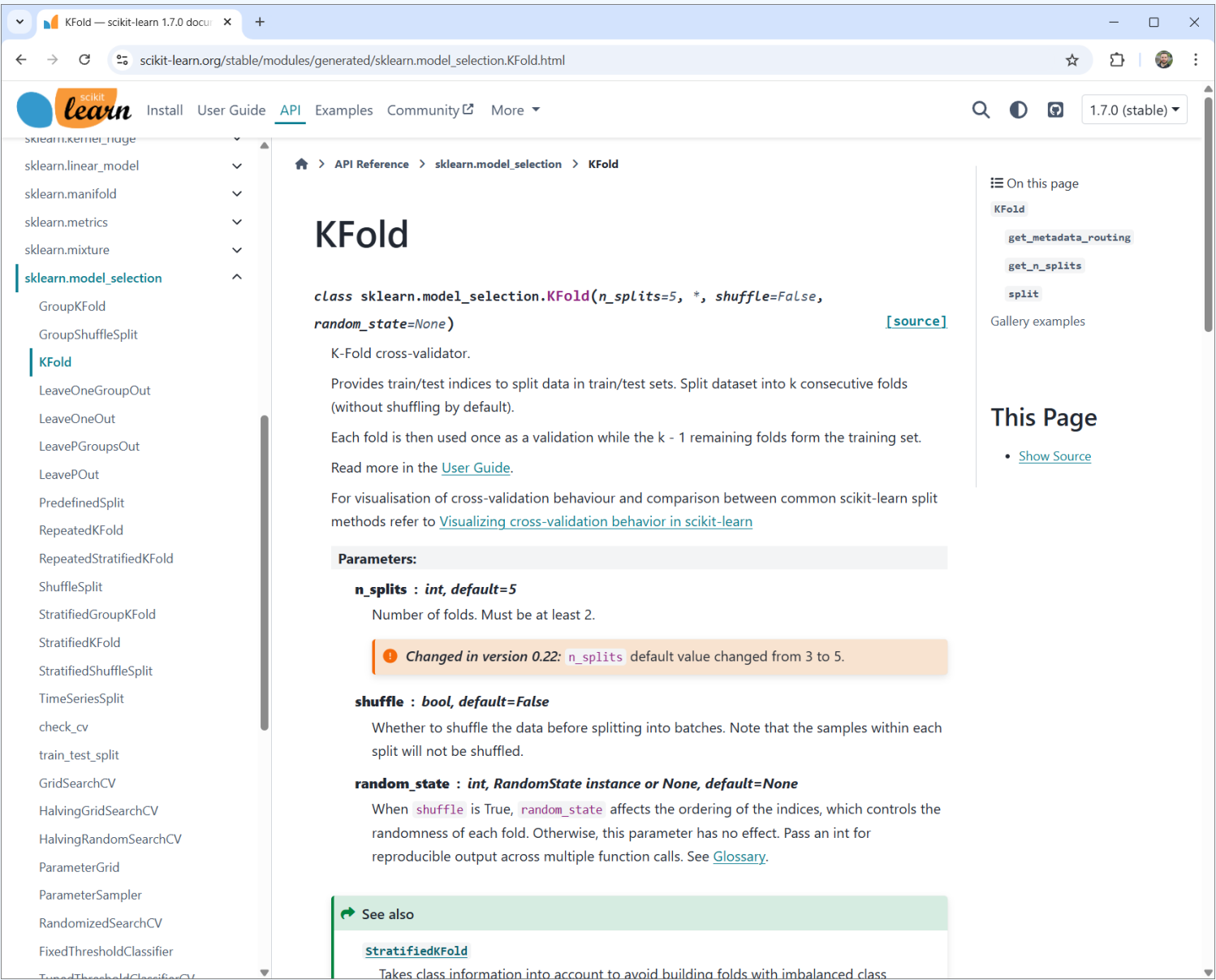
- O custo computacional é K vezes maior do que treinar o modelo uma única vez.
- Pode não ser ideal para *datasets* desbalanceados, pois a divisão aleatória pode criar *folds* sem representantes de uma classe minoritária.

VALIDAÇÃO CRUZADA (CV)

K-FOLD



Fonte: <https://www.baeldung.com/cs/cross-validation-k-fold-loo>



VALIDAÇÃO CRUZADA (CV)

K-FOLD ESTRATIFICADO

- **Funcionamento:**

- Uma variação do K-Fold projetada especificamente para problemas de **classificação com datasets desbalanceados**.
- O Stratified K-Fold garante que cada fold tenha **aproximadamente a mesma proporção de amostras de cada classe** que o conjunto de dados original.

- **Pontos Positivos:**

- Todos os benefícios do K-Fold, mais a garantia de representatividade das classes.
- Leva a estimativas de performance mais confiáveis e com menor variância em problemas de classificação.

- **Pontos Negativos:**

- Não é aplicável a problemas de regressão (pois não existem "classes" para estratificar).

VALIDAÇÃO CRUZADA (CV)

K-FOLD ESTRATIFICADO



Fonte: <https://www.baeldung.com/cs/cross-validation-k-fold-loo>

StratifiedKFold — scikit-learn 1.7.0

scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html

scikit-learn

Install User Guide API Examples Community More

sklearn.kernel_ridge

sklearn.linear_model

sklearn.manifold

sklearn.metrics

sklearn.mixture

sklearn.model_selection

GroupKFold

GroupShuffleSplit

KFold

LeaveOneGroupOut

LeaveOneOut

LeavePGroupsOut

LeavePOut

PredefinedSplit

RepeatedKFold

RepeatedStratifiedKFold

ShuffleSplit

StratifiedGroupKFold

StratifiedKFold

StratifiedShuffleSplit

TimeSeriesSplit

check_cv

train_test_split

GridSearchCV

HalvingGridSearchCV

HalvingRandomSearchCV

ParameterGrid

ParameterSampler

RandomizedSearchCV

FixedThresholdClassifier

API Reference

sklearn.model_selection

StratifiedKFold

StratifiedKFold

`class sklearn.model_selection.StratifiedKFold(n_splits=5, *, shuffle=False, random_state=None)` [\[source\]](#)

Class-wise stratified K-Fold cross-validator.

Provides train/test indices to split data in train/test sets.

This cross-validation object is a variation of KFold that returns stratified folds. The folds are made by preserving the percentage of samples for each class in `y` in a binary or multiclass classification setting.

Read more in the [User Guide](#).

For visualisation of cross-validation behaviour and comparison between common scikit-learn split methods refer to [Visualizing cross-validation behavior in scikit-learn](#)

Note

Stratification on the class label solves an engineering problem rather than a statistical one. See [Cross-validation iterators with stratification based on class labels](#) for more details.

Parameters:

n_splits : `int`, `default=5`

Number of folds. Must be at least 2.

Changed in version 0.22:

`n_splits` default value changed from 3 to 5.

shuffle : `bool`, `default=False`

Whether to shuffle each class's samples before splitting into batches. Note that the samples within each split will not be shuffled.

random_state : `int`, `RandomState` instance or `None`, `default=None`

On this page

StratifiedKFold

`get_metadata_routing`

`get_n_splits`

`split`

Gallery examples

This Page

Show Source

VALIDAÇÃO CRUZADA (CV)

LEAVE-ONE-OUT

▪ Funcionamento:

- _ É um caso extremo de K-Fold onde o número de folds, K , é igual ao número total de amostras, N , no conjunto de dados.
- _ Tem uso indicado com ***datasets* muito pequenos**, onde maximizar os dados de treino em cada passo é crucial e o custo computacional é viável.

▪ Pontos Positivos:

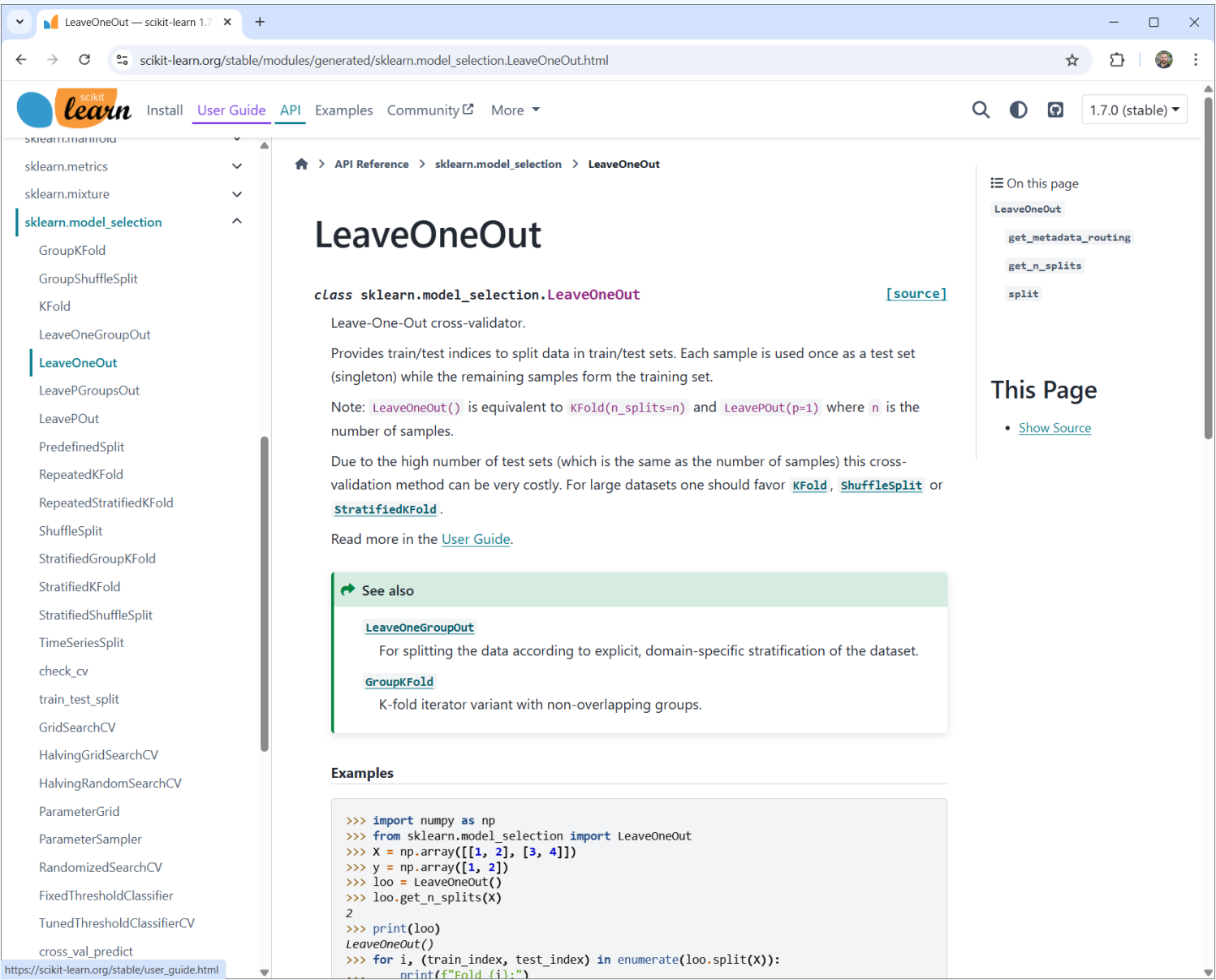
- _ Usa o máximo de dados possível para o treino em cada iteração, levando a uma estimativa de erro com baixo viés.
- _ Não há aleatoriedade no processo; o resultado será sempre o mesmo.

▪ Pontos Negativos:

- _ **Custo computacional extremo!** É inviável para *datasets* com mais do que algumas centenas de amostras.
- _ A estimativa de performance pode ter alta variância, pois os modelos treinados em cada iteração são quase idênticos (diferem por apenas um ponto), tornando as avaliações muito correlacionadas.

VALIDAÇÃO CRUZADA (CV)

LEAVE-ONE-OUT



LeaveOneOut

`class sklearn.model_selection.LeaveOneOut`

Leave-One-Out cross-validator.

Provides train/test indices to split data in train/test sets. Each sample is used once as a test set (singleton) while the remaining samples form the training set.

Note: `LeaveOneOut()` is equivalent to `KFold(n_splits=n)` and `LeavePOut(p=1)` where `n` is the number of samples.

Due to the high number of test sets (which is the same as the number of samples) this cross-validation method can be very costly. For large datasets one should favor `KFold`, `ShuffleSplit` or `StratifiedKFold`.

Read more in the [User Guide](#).

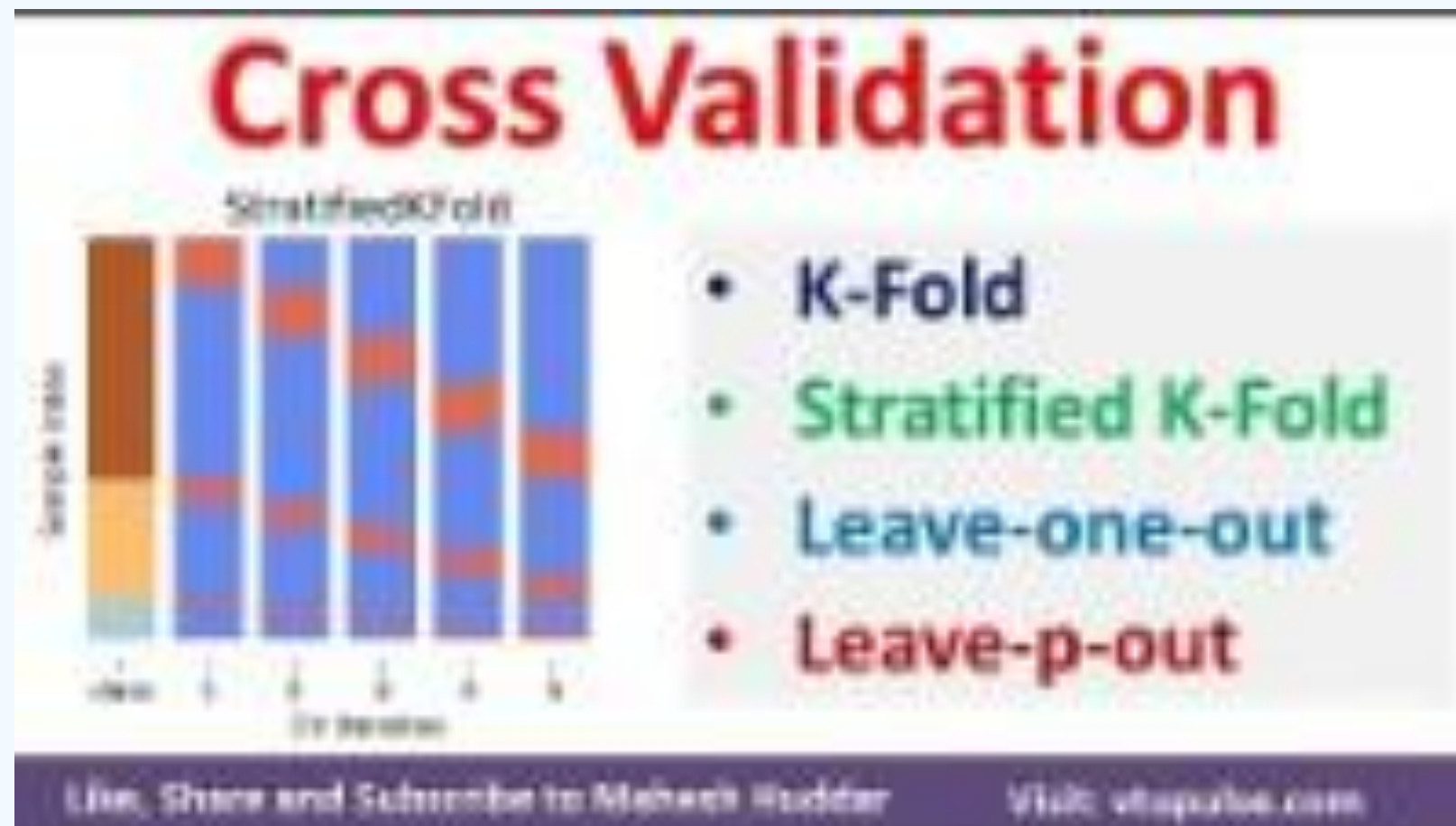
See also

- `LeaveOneGroupOut`: For splitting the data according to explicit, domain-specific stratification of the dataset.
- `GroupKFold`: K-fold iterator variant with non-overlapping groups.

Examples

```
>>> import numpy as np
>>> from sklearn.model_selection import LeaveOneOut
>>> X = np.array([[1, 2], [3, 4]])
>>> y = np.array([1, 2])
>>> loo = LeaveOneOut()
>>> loo.get_n_splits(X)
2
>>> print(loo)
LeaveOneOut()
>>> for i, (train_index, test_index) in enumerate(loo.split(X)):
...     print(f"fold {i}:")
```

VALIDAÇÃO CRUZADA (CV)



TÓPICO 4

CURVAS DE GAIN E LIFT: AVALIANDO O RETORNO POTENCIAL DE UM MODELO

CURVAS DE GAIN E LIFT

POR QUE GAIN E LIFT?

- **Motivação:**

- Acurácia, F1 e AUC-ROC dizem o quão bem o modelo separa as classes, mas não respondem à pergunta de negócio: "quanto eu ganho ao usar o modelo em vez de agir aleatoriamente?"
- Na prática os recursos são limitados: só se aborda uma fração da base (campanha, cobrança, auditoria).

- **Ideia central:**

- Ordenar as observações pela probabilidade prevista e verificar quanto da classe positiva é capturado ao abordar apenas os primeiros X% da base.
- Gain e Lift traduzem a performance em retorno e priorização — usuais em marketing, crédito, churn e fraude.

CURVAS DE GAIN E LIFT

ANÁLISE POR
DECIS

CURVA DE GAIN

CURVA DE LIFT

CURVAS DE GAIN E LIFT

ANÁLISE POR DECIS (DECILE ANALYSIS)

▪ Funcionamento:

- Calcula-se a probabilidade prevista para cada observação do conjunto de teste.
- Ordenam-se as observações em ordem decrescente de probabilidade.
- Divide-se a base ordenada em 10 grupos iguais (decis): o decil 1 reúne os 10% de maior score.
- Para cada decil calcula-se: positivos, taxa de resposta, positivos acumulados e % acumulado do total de positivos.
- Interpretação:
 - Em um bom modelo os positivos se concentram nos primeiros decis e a taxa de resposta decresce do decil 1 ao 10.
 - A tabela de decis é a base de cálculo das curvas de Gain e de Lift.

CURVAS DE GAIN E LIFT

CURVA DE GAIN (GAINS CHART)

▪ Definição:

- Gráfico que plota, no eixo X, o percentual acumulado da base abordada (decis) e, no eixo Y, o percentual acumulado dos positivos capturados — que é exatamente o recall naquele ponto de corte.

▪ Interpretação:

- A diagonal representa o modelo aleatório: abordando 30% da base, captura-se 30% dos positivos.
- Se a curva do modelo indica 70% no ponto de 30% da base, significa que abordando os 30% de maior score capturam-se 70% de todos os clientes que responderiam.
- Quanto mais a curva se afasta da diagonal e se aproxima do canto superior esquerdo, melhor o poder de ordenação do modelo.

▪ Uso prático:

- Definir o ponto de corte da campanha: até onde vale a pena avançar na base antes que o ganho marginal fique pequeno.

CURVAS DE GAIN E LIFT

CURVA DE LIFT (LIFT CHART)

- **Definição:**

- O Lift é a razão entre o resultado obtido com o modelo e o esperado sem modelo (seleção aleatória), no mesmo percentual da base.

- **Fórmula:**

- $\text{Lift} = (\% \text{ acumulado de positivos}) / (\% \text{ acumulado da base abordada})$

- **Interpretação:**

- Lift = 3 no primeiro decil: aquele grupo contém 3 vezes mais positivos que uma amostra aleatória de mesmo tamanho.
 - Lift = 1 equivale ao acaso; abaixo de 1, pior que o aleatório naquela faixa.
 - A curva começa alta e converge para 1 ao abordar 100% da base.

- **Pontos de atenção:**

- O Lift depende da prevalência da classe positiva, dificultando comparações entre bases diferentes.

CURVAS DE GAIN E LIFT

GAIN/LIFT vs. CURVA ROC

- **Diferença fundamental:**

- A ROC plota Taxa de Verdadeiros Positivos contra Taxa de Falsos Positivos e mede a discriminação do modelo, sendo insensível ao desbalanceamento.
- A curva de Gain plota positivos capturados contra o % da base abordada — o eixo X inclui positivos e negativos, logo depende da prevalência.

- **Quando usar cada uma:**

- ROC/AUC: comparar e selecionar modelos.
- Gain/Lift: comunicar ao negócio, dimensionar campanhas e definir quanto da base abordar.
- Boas práticas: avaliar em dados de teste e sempre comparar com a linha aleatória.

VALIDAÇÃO CRUZADA (CV)

Spotfire Community

ForumsEventsExchangeIndustriesMore

Existing user? Sign InSign Up

HomeArticlesSpotfireGains vs ROC curves. Do you understand the difference?

Search...

Gains vs ROC curves. Do you understand the difference?

Analytics

ShareFollowers

Table of contents

Gains chart

Confusion matrix

ROC curve

Usage

So, what is the difference?

Lift chart

Useful links

Gains chart

Typically called a Cumulative Gains Chart it can be simply explained by the following example:

For simplicity let's assume we have 1000 customers. If we run an advertising campaign for all our customers, we might find that 30% (300 out of 1000) will respond and buy our new product.

Marketing to all our customers could be one strategy for running a campaign. But this is not the optimum use of our marketing dollars, especially for large customer bases. Therefore we would like to have a better way of running this advertising campaign so that instead of targeting all our customer base, we target only to those customers with a high probability of responding positively to the campaign. This will, firstly lower the cost of the campaign and, secondly (and maybe more importantly) we will not disturb those customers with advertising who have no interest in our new product.

This is where predictive classification models come in. There are lots of different models, but no matter which one we use, we can still evaluate the results of our model by using Cumulative Gains Charts. If we have historical data on the reactions of customers to past campaigns, we can use the data to build a model that predicts, if a particular customer will respond by buying the product or not. The results of such a model are typical, for each customer, the probability of a positive and negative reaction from the customer. We can sort customers according to the probability of a positive reaction to the

